



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

**Лабораторная работа № 6
по курсу «Алгоритмы компьютерной графики»
«Построение реалистичных изображений»**

Студент группы ИУ9-42Б Ивенкова О. В.

Преподаватель Цалкович П. А.

Москва 2025

1 Задача

а. Базовой лабораторной работой является лабораторная работа №3 (модельно-видовые преобразования и преобразования проецирования).

б. Определить параметры модели освещения OpenGL (свойства источника света, свойства материалов (поверхностей), характеристики глобальной модели освещения).

в. Исследовать один из методов повышения реалистичности получаемых изображений сцены.

- Персональный вариант: А3. влияние параметров (компонентов модели освещения) источника света;

г. Реализовать один из алгоритмов анимации.

- Персональный вариант: Б1. моделирование равноускоренного падения (с заданной начальной скоростью) при условии абсолютно упругого соударения с горизонтальной поверхностью;

д. Реализовать наложение текстуры (загрузка из файла *.bmp или процедурная генерация) с возможностью отключения.

- Персональный вариант: В1. использование текстуры для определения интенсивности поверхности;

2 Основная теория

Средства повышения реалистичности изображения

- использование освещения:
 - модели освещения (диффузное, направленное);
 - отражение (диффузное, зеркальное) – свойства материала;
 - преломление;
 - прозрачность (blending, комбинация цветов в различных режимах), цвет;
 - построение теней;
- фактуры (текстуры) – нанесение некоторого изображения на поверхность или внесение возмущения в параметры поверхности;
- мультитекстурирование (multi-texturing), наложение карты окружения (environment mapping);
- движение, анимация;
- использование фракталов, шумовых функций (шум Перлина – Perlin noise);
- ...

Движение

- синхронизация (timing) - связь реального времени с частотой смены кадра;
- спецификация движения:
 - прямая спецификация движения: явное задание матриц геометрических преобразований (углов движения и векторов перемещения) объектов сцены, или задание аппроксимирующих кривых;
 - целенаправленная спецификация: определение набора действий и поддействий, приводящих к некоторому результату;
 - спецификация на основе физических законов:
 - кинематическое описание (задание положения, скорости, ускорения; или задание траектории);
 - обратное кинематическое описание (определение параметров движения через заданное начальное и конечное положение объекта);
 - динамическое описание (определение сил, использование законов сохранения) – физическое моделирование;
 - обратное динамическое описание (определение действующих сил через заданное начальное и конечное положение объекта).

Абсолютно упругие соударения

- при абсолютно упругих соударениях:
 - не происходит деформации объектов при соударении;
 - применимы законы сохранения импульса и энергии;
- центральное соударение шаров:
 - законы сохранения импульса и массы:

$$v_{1i} \cdot m_1 + v_{2i} \cdot m_2 = v_{1f} \cdot m_1 + v_{2f} \cdot m_2$$

$$m_1 \cdot v_{1i}^2 + m_2 \cdot v_{2i}^2 = m_1 \cdot v_{1f}^2 + m_2 \cdot v_{2f}^2$$
 - решение:

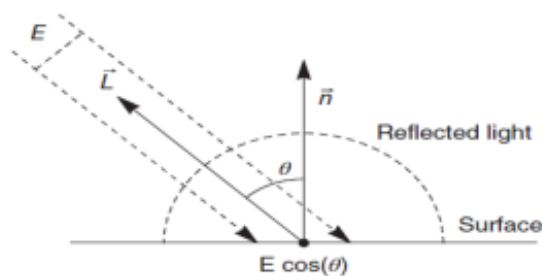
$$v_{1f} = (2 \cdot m_2 \cdot v_{2i} + (m_1 - m_2) \cdot v_{1i}) / (m_1 + m_2)$$

$$v_{2f} = (2 \cdot m_2 \cdot v_{1i} + (m_2 - m_1) \cdot v_{2i}) / (m_1 + m_2)$$
- соударение шара с наклонной плоскостью:
 - $y + k \cdot x + b = 0$
 - $(x - x_c)^2 + (y - y_c)^2 = R^2$
 - + 2 параметрических уравнения перемещения центра
 - + $D = 0$ (касание)
- соударение шара с горизонтальной / вертикальной плоскостью:
 - угол падения равен углу отражения;
 - модуль скорости после соударения не изменяется.

Освещение

- световая энергия, падающая на поверхность, может быть:
 - поглощена (absorption);
 - отражена (reflection);
 - пропущена (transmission);
- количество поглощенной, отраженной или пропущенной энергии зависит от длины волны света (соответственно, изменяется распределение энергии и объект выглядит цветным, а цвет объекта определяется поглощаемыми длинами волн);
- если объект поглощает весь падающий свет, то он невидим и называется *абсолютно черным телом*;
- свойства отраженного света зависят:
 - от строения, направления и формы источника света;
 - от ориентации и свойств поверхности (диффузное и зеркальное отражение).

- Light



Диффузное отражение (diffuse scattering): закон косинусов Ламберта

- диффузное отражение света происходит, когда свет как бы проникает под поверхность объекта, поглощается, а затем вновь испускается;
- диффузно отраженный свет рассеивается равномерно по всем направлениям, следовательно, положение наблюдателя не имеет значения;
- свет точечного источника отражается от идеального рассеивателя по закону косинусов Ламберта (Lambert): интенсивность отраженного света пропорциональна косинусу угла между направлением света и нормалью к поверхности:

$$I = I_l k_d \cos \theta$$

- где I — интенсивность отраженного света,
 - I_l — интенсивность точечного источника,
 - k_d — коэффициент диффузного отражения ($0 \leq k_d \leq 1$),
 - θ — угол между направлением света и нормалью к поверхности
- цвет определяется свойствами материала;
 - коэффициент диффузного отражения k_d зависит от материала и длины волны света, но в простых моделях освещения обычно считается постоянным.

Текстуры

- наложение текстуры выполняется с помощью функции отображения текстурного и объектного координатных пространств:
 - задается текстурная функция (texture map) $texture(s, t)$ в так называемом *текстурном пространстве* (texture space), которая генерирует значения цвета или яркости для каждого значения $s, t \in [0, 1]$;
 - производится отображение на требуемую поверхность, заданную в объектном пространстве $(s, t) \rightarrow (x, y, z)$;
 - производится стандартное преобразование для вывода на экран;
- на практике может решаться обратная задача: определение текстурных координат, соответствующих заданному значению экранных;
- значения текстурной функции могут использоваться:
 - для задания интенсивности грани;
 - для определения коэффициентов отражения.

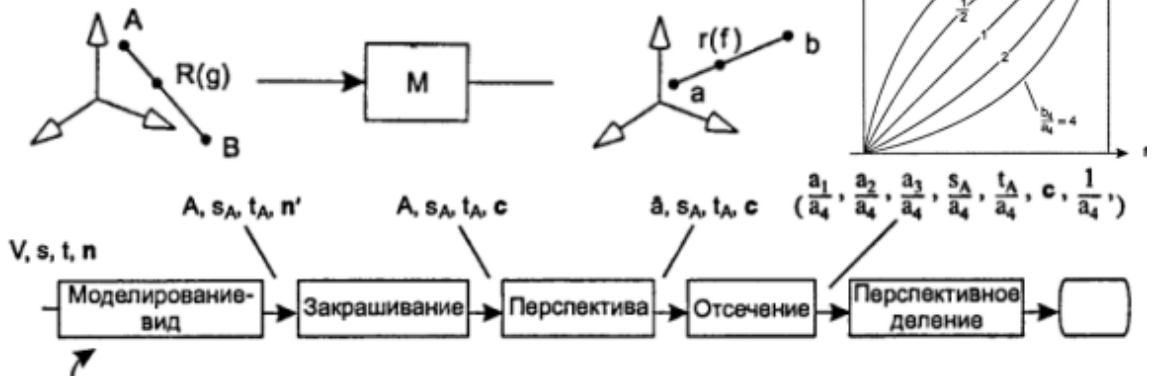


Текстуры и интерполяция

- при растровой развертке линейная интерполяция текстурных координат с равномерным шагом, в общем случае, приводит к некорректным результатам (равные шаги по спроецированной грани не соответствуют равным шагам по трехмерной грани);
- гиперболическая интерполяция – учет перспективного преобразования при отображении текстурных координат в объектное пространство (обеспечивается конвейером).



$$\frac{1}{z} = (1 - \alpha) \frac{1}{z_1} + \alpha \frac{1}{z_2} \quad \frac{u}{z} = (1 - \alpha) \frac{u_1}{z_1} + \alpha \frac{u_2}{z_2}$$



3 Практическая реализация

Исходный код программы представлен в листинге 1.

Листинг 1: Реализация файла main.py

```

1 import glfw
2 from OpenGL.GL import *
3 from OpenGL.GLUT import *
4 from OpenGL.GLU import *
5 import numpy as np
6 from PIL import Image
7
8 angle_x, angle_y = 0, 0
9 scale = 0.7
10 wireframe = False
11 light_diffuse = [0.2, 0.2, 0.2, 1.0]
12 light_position_toggle = False
13 light_diffuse_toggle = False
14 light_specular_toggle = False
15
16 gravity = -9.8
17 y_pos = 1.0
18 y_velocity = 2.5

```



```

19 last_time = glfw.get_time()
20 use_texture = True
21
22 a, b, c = 1.0, 0.6, 0.8
23 num_lat = 10
24 num_lon = 20
25
26 def parametric_ellipsoid(theta, phi):
27     x = a * np.cos(theta) * np.cos(phi)
28     y = b * np.sin(theta)
29     z = c * np.cos(theta) * np.sin(phi)
30     return x, y, z
31
32 def draw_parametric_ellipsoid():
33     glBegin(GL_TRIANGLES)
34
35     for i in range(num_lat):
36         theta1 = -np.pi / 2 + np.pi * i / num_lat
37         theta2 = -np.pi / 2 + np.pi * (i + 1) / num_lat
38
39         for j in range(num_lon):
40             phi1 = 2 * np.pi * j / num_lon
41             phi2 = 2 * np.pi * (j + 1) / num_lon
42
43             v1 = parametric_ellipsoid(theta1, phi1)
44             v2 = parametric_ellipsoid(theta2, phi1)
45             v3 = parametric_ellipsoid(theta2, phi2)
46             v4 = parametric_ellipsoid(theta1, phi2)
47
48             if use_texture:
49                 glTexCoord2f(j / num_lon, i / num_lat)
50                 glVertex3fv(v1)
51
52             if use_texture:
53                 glTexCoord2f(j / num_lon, (i + 1) / num_lat)
54                 glVertex3fv(v2)
55
56             if use_texture:
57                 glTexCoord2f((j + 1) / num_lon, (i + 1) / num_lat)
58                 glVertex3fv(v3)
59
60             if use_texture:
61                 glTexCoord2f(j / num_lon, i / num_lat)
62                 glVertex3fv(v4)
63
64             if use_texture:

```

```

65         glTexCoord2f((j + 1) / num_lon, (i + 1) / num_lat)
66         glVertex3fv(v3)
67
68         if use_texture:
69             glTexCoord2f((j + 1) / num_lon, i / num_lat)
70             glVertex3fv(v4)
71     glEnd()
72
73 def draw_moving_ellipsoid():
74     glMatrixMode(GL_MODELVIEW)
75     glLoadIdentity()
76     glRotatef(angle_x, 1, 0, 0)
77     glRotatef(angle_y, 0, 1, 0)
78
79     glPushMatrix()
80     glTranslatef(0, y_pos, 0)
81     glScalef(scale, scale, scale)
82     glColor3f(75/255, 0, 130/255)
83     glMaterialfv(GL_FRONT_AND_BACK, GL_AMBIENT, [0.2, 0.2, 0.4, 1.0])
84     glMaterialfv(GL_FRONT_AND_BACK, GL_DIFFUSE, [0.3, 0.3, 0.8, 1.0])
85     glMaterialfv(GL_FRONT_AND_BACK, GL_SPECULAR, [1.0, 1.0, 1.0, 1.0])
86     glMaterialf(GL_FRONT_AND_BACK, GL_SHININESS, 50.0)
87     draw_parametric_ellipsoid()
88     glPopMatrix()
89
90 def setup_lighting():
91     glEnable(GL_LIGHTING)
92     glEnable(GL_LIGHT0)
93     glShadeModel(GL_SMOOTH)
94
95     glLightModelfv(GL_LIGHT_MODEL_AMBIENT, [0.5, 0.5, 0.5, 1.0])
96
97     glLightfv(GL_LIGHT0, GL_POSITION, [2.0, 2.0, 2.0, 1.0])
98     glLightfv(GL_LIGHT0, GL_DIFFUSE, [1.0, 1.0, 1.0, 1.0])
99     glLightfv(GL_LIGHT0, GL_SPECULAR, [1.0, 1.0, 1.0, 1.0])
100
101     glMaterialfv(GL_FRONT_AND_BACK, GL_AMBIENT, [0.2, 0.2, 0.4, 1.0])
102     glMaterialfv(GL_FRONT_AND_BACK, GL_DIFFUSE, [0.3, 0.3, 0.8, 1.0])
103     glMaterialfv(GL_FRONT_AND_BACK, GL_SPECULAR, [1.0, 1.0, 1.0, 1.0])
104     glMaterialf(GL_FRONT_AND_BACK, GL_SHININESS, 50.0)
105
106 def update_light():
107     global light_position_toggle, light_diffuse_toggle,
108     light_specular_toggle
109
110     if light_position_toggle:

```



```

110         glLightfv(GL_LIGHT0, GL_POSITION, [-2.0, -2.0, 2.0, 1.0])
111     else:
112         glLightfv(GL_LIGHT0, GL_POSITION, [2.0, 2.0, 2.0, 1.0])
113
114     if light_diffuse_toggle:
115         glLightfv(GL_LIGHT0, GL_DIFFUSE, [0.2, 1.0, 0.2, 1.0])
116     else:
117         glLightfv(GL_LIGHT0, GL_DIFFUSE, [1.0, 1.0, 1.0, 1.0])
118
119     if light_specular_toggle:
120         glLightfv(GL_LIGHT0, GL_SPECULAR, [0.0, 0.0, 0.0, 1.0])
121     else:
122         glLightfv(GL_LIGHT0, GL_SPECULAR, [1.0, 1.0, 1.0, 1.0])
123
124 def update_motion():
125     global y_pos, y_velocity, last_time
126
127     current_time = glfw.get_time()
128     delta_time = current_time - last_time
129     last_time = current_time
130
131     y_velocity += gravity * delta_time
132     y_pos += y_velocity * delta_time
133
134     if y_pos <= -1.0:
135         y_pos = -1.0
136         y_velocity = -y_velocity
137
138 def load_texture(path):
139     image = Image.open(path)
140     image = image.transpose(Image.FLIP_TOP_BOTTOM)
141     img_data = image.convert("RGB").tobytes()
142
143     texture_id = glGenTextures(1)
144     glBindTexture(GL_TEXTURE_2D, texture_id)
145
146     glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, image.width, image.height,
147                 0, GL_RGB, GL_UNSIGNED_BYTE, img_data)
148
149     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR)
150     return texture_id
151
152 def display():
153     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
154     draw_moving_ellipsoid()
155     glfw.swap_buffers(window)

```

```

156
157     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
158     draw_moving_ellipsoid()
159     glfw.swap_buffers(window)
160
161 def key_callback(window, key, scancode, action, mods):
162     global angle_x, angle_y, wireframe, scale, use_texture,
light_position_toggle, light_diffuse_toggle, light_specular_toggle
163     if action == glfw.PRESS or action == glfw.REPEAT:
164         if key == glfw.KEY_UP:
165             angle_x -= 5
166         elif key == glfw.KEY_DOWN:
167             angle_x += 5
168         elif key == glfw.KEY_LEFT:
169             angle_y -= 5
170         elif key == glfw.KEY_RIGHT:
171             angle_y += 5
172         elif key == glfw.KEY_EQUAL:
173             scale += 0.1
174         elif key == glfw.KEY_MINUS:
175             scale -= 0.1
176         elif key == glfw.KEY_W:
177             wireframe = not wireframe
178             glPolygonMode(GL_FRONT_AND_BACK, GL_LINE if wireframe else
GL_FILL)
179         elif key == glfw.KEY_T:
180             use_texture = not use_texture
181         if use_texture:
182             glEnable(GL_TEXTURE_2D)
183         else:
184             glDisable(GL_TEXTURE_2D)
185         elif key == glfw.KEY_1:
186             light_position_toggle = not light_position_toggle
187             update_light()
188         elif key == glfw.KEY_2:
189             light_diffuse_toggle = not light_diffuse_toggle
190             update_light()
191         elif key == glfw.KEY_3:
192             light_specular_toggle = not light_specular_toggle
193             update_light()
194
195 def scroll_callback(window, xoffset, yoffset):
196     global scale
197     scale += 0.1 if yoffset > 0 else -0.1
198
199 def init():

```

```

200     glEnable(GL_DEPTH_TEST)
201     glEnable(GL_TEXTURE_2D)
202     texture_id = load_texture(r"C:\Go\projects\Bmstu-projects\Grafic\
lab6\texture.bmp")
203     glTexEnvf(GL_TEXTURE_ENV, GL_TEXTURE_ENV_MODE, GL_MODULATE)
204     setup_lighting()
205
206     if not glfw.init():
207         raise Exception("GLFW can't be initialized")
208
209     window = glfw.create_window(800, 600, "lab6", None, None)
210     if not window:
211         glfw.terminate()
212         raise Exception("GLFW window can't be created")
213
214     glfw.make_context_current(window)
215
216     glfw.set_key_callback(window, key_callback)
217     glfw.set_scroll_callback(window, scroll_callback)
218     init()
219
220     while not glfw.window_should_close(window):
221         update_motion()
222         display()
223         glfw.poll_events()
224
225     glfw.terminate()

```

Результат запуска представлен на рисунках 1– 2.

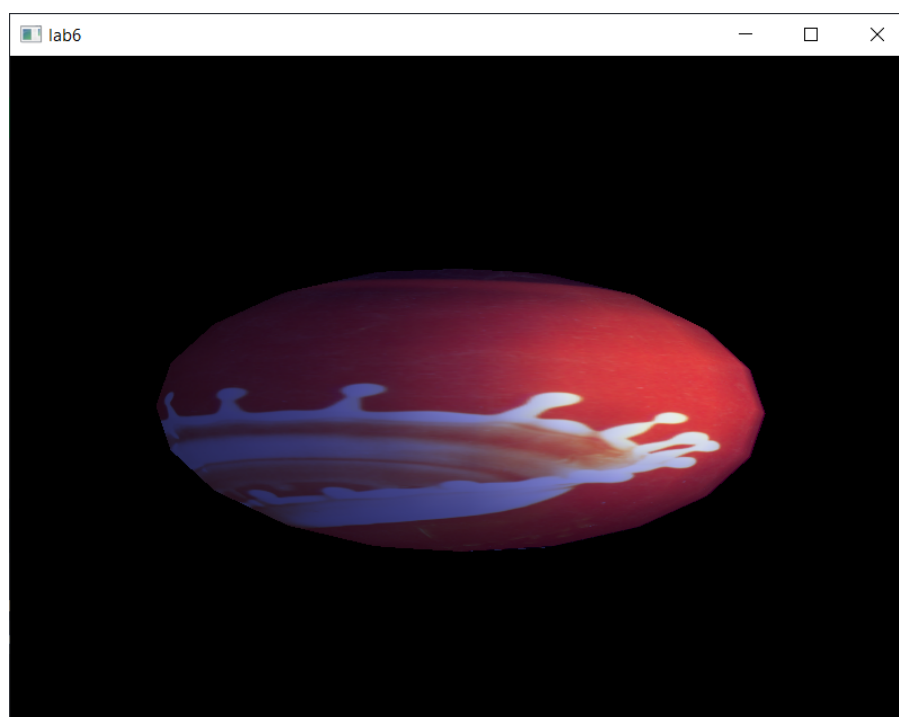


Рис. 1 — Эллипсоид с наложением текстуры

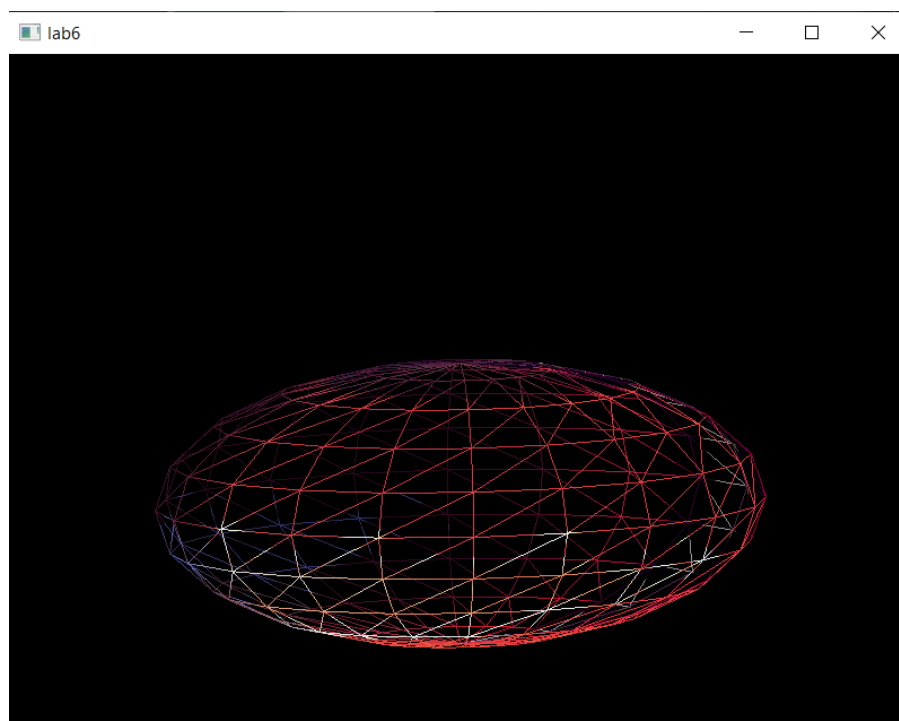


Рис. 2 — Каркасное отображение эллипсоида

4 Заключение

В ходе выполнения лабораторной работы были реализованы основные этапы построения реалистичных изображений. Базой послужила сцена с параметрическим эллипсоидом (лабораторная работа №3), дополненная моделью освещения с настраиваемыми компонентами источника света и материалами поверхности.

Проведено исследование влияния диффузной и зеркальной компонент, а также положения источника света на визуальные характеристики объекта.

Реализован алгоритм анимации — моделирование равноускоренного падения с абсолютно упругим соударением, что позволило имитировать физическое поведение тела. При абсолютно упругом соударении эллипсоида с горизонтальной поверхностью реализовано отражение с сохранением модуля скорости. Направление скорости изменяется на противоположное. Энергия системы сохраняется, в расчётах используется закон сохранения механической энергии.

Также внедрено наложение текстуры, влияющей на интенсивность поверхности посредством модуляции с освещением, с возможностью включения и отключения по команде пользователя.

В результате была получена динамичная сцена с реалистичным освещением и текстурированием, демонстрирующая основные приёмы повышения качества визуализации в OpenGL.